

PrograUDD Ayudantía 06

Instrucciones: Resuelva con un compañero/a la siguiente lista de ejercicios. Recuerde discutir la estrategia para resolver cada problema, y luego escribir el código solución. Suba su solución como un archivo `ay06_apellido1apellido2.py` a Canvas, Sección Tareas > Ayudantías.

Para cada ejercicio escriba un comentario al inicio de su código declarando la entrada y salida de cada uno de los problemas.

1. Contactos.

1a. (1.0 pt) Cree una función que, a partir de 4 listas:

- lista de 6 nombres (ej. ['Pedro','Daniela','Juan','Paula','Diego','Javiera'])
- lista de 5 iniciales de apellido (ej. ['A.', 'C.', 'F.', 'G.', 'S.'])
- lista de Años de Nacimiento (ej. [1995, 1996, ..., 2002])
- lista de 4 Comunas (ej. ['Santiago', 'Concepcion', 'Las Condes', 'Talcahuano']),

genera un diccionario consistente en $n=15$ personas ficticias generadas al azar a partir de estas listas.

El diccionario debe tener como llave una combinación aleatoria entre algún nombre y algún apellido (ej. 'Pedro A.'), con cada elemento del diccionario una tupla de dos elementos: año de nacimiento y la comuna donde vive. (ej. (1999,'Santiago'))

Un ejemplo:

```
d1={'Daniela F.':(1995,'Santiago'), ...}
```

1b. *Función Contactos* (1.5 pts). Cree la función `contactos(nombre, d1)` que recibe el nombre de una persona y el diccionario de la pregunta anterior y retorne los nombres de todas las otras personas que viven en la misma comuna.

Hint: Una estrategia es obtener la comuna de la persona y luego iterar sobre las llaves, para comparar las comunas.

2. Naipes

2a. (0.7pt) Programe la función `naipes()` que simula barajar un mazo de naipes inglés al azar al generar una lista con las 52 cartas, donde las cartas por (número,pinta) no se repiten. Los números van de (2,3, ... 14), donde **J** es 11, **Q** es 12, **K** es 13 y **A** es 14.

2b. (0.8pt) Programe la función `repartir(n)`, donde n es el número de jugadores ($n \leq 8$) donde se reparten 5 cartas a cada jugador. El modo de repartir es la 1a carta al 1er jugador, 2a al 2o y así sucesivamente donde la $(n+1)$ carta se reparte al 1er jugador. La función debiera entregar un listado de n elementos, donde cada elemento es una lista (o tupla) de las 5 cartas correspondientes a cada jugador.

2c. (1.5pts) Programe la función `ganador(lista)`, donde la variable de entrada es la salida de la parte 2b, y entrega el jugador que tiene la carta más alta. En caso que haya un empate de la carta más alta entre 1 o más jugadores, se compara la carta siguiente de los que empataron y así sucesivamente si se empatara con las siguientes cartas.

3. Levadura (1.pt). El ácido desoxirribonucleico (ADN), contiene las instrucciones genéticas usadas en el desarrollo y funcionamiento de todos los organismos vivos y algunos virus. El ADN esta formado por nucleótidos, los cuales contienen bases nitrogenadas y otras moléculas. Los nucleótidos se organizan en cadenas que se repiten para formar el ADN.

El archivo `gen_levadura.txt` contiene la secuencia de ADN para el gen de la levadura. Este describe la secuencia genética cada 60 nucleótidos por fila, donde el número que inicia cada línea del archivo corresponde al número del nucleótido que inicia la secuencia de la fila, mientras que letras `a`, `t`, `c` y `g` subsiguientes corresponden a las bases nitrogenadas adenina, timina, citosina y guanina respectivamente, que son parte de cada nucleótido.

Utilizando la información anterior cree una función que permita contar el número de bases nitrogenadas de una secuencia de ADN como la del archivo `gen_levadura.txt`. La función debe retornar un diccionario, donde la clave es la letra correspondiente a la base nitrogenada (`a`, `t`, `c` y `g`), y el valor corresponda a la cantidad de veces que aparece cada base en la secuencia.